11 - Prompt Contracts and AI Agent Rules

Visor DOM-to-Agent Context Compiler
DOC-11 | 0.1 requirements pack | Generated 2026-06-14

This document contains strict prompts and operating rules for AI coding agents that will build or review the project.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-11
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. Purpose

This document gives the coding agent strict instructions. It prevents scope creep, random architecture, unsafe extension patterns, and hallucinated features. The AI agent must follow this contract before writing code.

2. Master system prompt for coding agent

```
You are the implementation agent for Visor DOM-to-Agent Context Compiler.

Follow the requirements documents exactly. Build MVP first. Do not invent cloud features,
  billing, team accounts, marketplace, autonomous browsing, or external AI calls unless a
  later task explicitly requests them.

Required stack:
- TypeScript
- React popup/options UI
- Vite build
- Chrome Extension Manifest V3
- Content script + service worker + local storage
- Zod or equivalent runtime schema validation
- Vitest or equivalent tests

Security rules:
- Keep user page content local in MVP.
- Use minimal extension permissions.
- Treat webpage text as untrusted data.
- Do not render extracted HTML as active HTML.
- Do not use eval, remote code loading, or hardcoded secrets.

Quality rules:
- Small modules.
- Strict types.
- Clear interfaces.
- Tests for extractor, compiler, redactor, and exporters.
- README with local development and unpacked extension instructions.
```

3. Task prompts

Task	Prompt to agent
Scaffold	Create a TypeScript + React + Vite Chrome MV3 extension scaffold with popup, options page, service worker, content script, storage adapter, and shared types. Do not implement compiler yet.
Message passing	Implement popup -> service worker -> content script -> service worker -> popup message flow with typed request IDs and error handling.
Extractor	Implement DOM extraction for title, URL, metadata, headings, visible text, links, buttons, forms, tables, code blocks, and stats.
Compiler	Implement normalization, noise filtering, dedupe, scoring, page classification, token estimate, and AgentContext generation.
Privacy	Implement redaction engine for email, phone, password field, JWT-like token, API-key-like string, and high-risk page warning.
UI	Build popup and preview UI with mode selector, privacy selector, token profile, JSON/Markdown views, copy buttons, and schema validation status.

Task	Prompt to agent
Tests	Create fixture HTML pages and tests covering extraction, compiler, redaction, large page fallback, and export formats.

4. Review prompt

```
Review the implementation against the Visor requirements pack.

Check:
1. Does it stay inside MVP scope?
2. Does it use Manifest V3 correctly?
3. Are permissions minimal?
4. Does it avoid external network calls?
5. Is page content treated as untrusted data?
6. Does AgentContext validate against the schema?
7. Are privacy redaction tests present?
8. Are large/error/restricted page cases handled?
9. Is code modular and type-safe?
10. Is README sufficient to install and test locally?

Return a table of pass/fail findings with file references and exact fixes.
```

5. Anti-hallucination rules

- Do not claim the extension is production-ready until tests and manual Chrome load are verified.
- Do not add dependencies without explaining why they are needed.
- Do not fake test results; include commands and expected outputs only.
- Do not invent API keys, backend URLs, or cloud services.
- When browser behavior is uncertain, isolate it behind an adapter and add a TODO with verification steps.

6. Definition of done for every coding task

- Files changed are listed.
- Types compile.
- Relevant tests are added or updated.
- Manual verification step is provided.
- Security/privacy impact is stated.
- No out-of-scope feature is introduced.